

SOC Inteligente

Detección y Respuesta ante Amenazas con Inteligencia Artificial

Dr. Maurice Frayssinet Delgado

¿QUÉ ES UN SOC?

(Security Operations Center)



El centro neurálgico de
la ciberseguridad corporativa

¿Cómo lo definen los expertos?

"Un SOC es un enfoque tecnológico convergente para la detección, investigación y respuesta ante amenazas (TDIR) mediante un conjunto de tecnologías de seguridad integradas." — Gartner (2026).

"Su misión es detectar, analizar y responder a incidentes de seguridad en tiempo real... mejorando la prevención." — IBM Security.

1. PERSONAS
(El Equipo)

Analistas, ingenieros de seguridad y expertos en respuesta a incidentes que operan 24/7.

2. PROCESOS
(La Estrategia)

Monitoreo continuo, detección de anomalías, investigación y contención rápida de amenazas.

3. TECNOLOGÍA
(Las Herramientas)

SIEM, IA, SOAR, XDR y firewalls para analizar datos masivos y automatizar la defensa.

LA HISTORIA DEL SOC

Camino Recorrido del SOC

(Security Operations Center)

Primeros Indicios
(1970s - 1980s)



Nacimiento de los SOCs
(Años 1990 - 2000)



SOC Centralizado y SIEM
(Años 2010)



La Era Cloud, SOAR y XDR
(Años 2020)



EL SOC EN LA ACTUALIDAD
IA generativa, SOC autónomo, operaciones predictivas y Zero Trust



IA generativa, SOC autónomo, operaciones predictivas y Zero Trust

Primeros Indicios (1970s - 1980s):
Monitoreo de redes primitivas y registros (Logs) básicos

El Nacimiento (1990s - 2000s):
Primeros equipos dedicados, firewalls e IDS básicos

Madurez (2010s):
Centralización con SIEM, detección de amenazas complejas y respuesta a incidentes

Evolución Rápida (2020-2024): Migración a la nube, automatización (SOAR) y detección proactiva (XDR)

EL FUTURO DEL SOC (2027+)
IA hiperautomatizada, operaciones cuánticas predictivas, sistemas totalments autónomos y ciberseguridad adaptativa global.



ROLES EN UN SOC

(Centro de Operaciones de Seguridad)

NIVEL 1: TRIAGE Y ALERTAS

Monitoreo 24/7 y filtrado inicial



Analista de Seguridad L1

- Monitoreo constante de alertas
- Filtrado de falsos positivos
- Escalado de incidentes reales

NIVEL 2: RESPUESTA A INCIDENTES

Investigación y contención



Analista de Seguridad L2 / Incident Responder

- Investigación a fondo de amenazas
- Contención de ataques activos
- Mitigación y recuperación

NIVEL 3: THREAT HUNTING Y FORENSE

Amenazas complejas y avanzadas



Threat Hunter / Especialista Forense (L3)

- Búsqueda proactiva de amenazas
- Análisis forense de ataques
- Análisis de malware avanzado



SOC Manager



Ingeniero de Seguridad

GESTIÓN Y SOPORTE

Liderazgo, mejora y optimización

- Gestión de incidentes y Reportes (Manager)
- Liderazgo y coordinación del equipo (Manager)
- Mantenimiento de herramientas y reglas (Ingeniero)

¡Un equipo coordinadora y especializado

¡Un equipo para proteger la infraestructura frente a cualquier frente menaize ciberamenaza!

EL SOC EN LA ERA DE LA IA

(La automatización y evolución de los roles de ciberseguridad)

NIVEL 1: TOTALMENTE AUTOMATIZADO

Agentes de IA Autónomos



Agentes de IA y bots autónomos

- Filtrado de ruido y falsos positivos
- Triaje automático de alertas 24/7
- Respuestas inmediatas automatizadas

FUSIÓN N2 Y N3: ANALISTAS AUMENTADOS

Investigación y caza asistida por IA



Analistas Aumentados Threat Hunters asistidos por IA

- Copilotos para investigación ultrarrápida
- Caza proactiva de amenazas complejas
- Mitigación y respuesta experta

NUEVO ROL: INGENIERO DE IA Y SEC_OPS

El director de la automatización



Ingeniero de IA y Automatización SecOps

- Programación y diseño de Playbooks
- Entrenamiento y supervisión de modelos IA
- Optimización de la infraestructura SOC



SOC Manager



Ingeniero de Seguridad

EL NUEVO ROL CLAVE

El director de la automatización

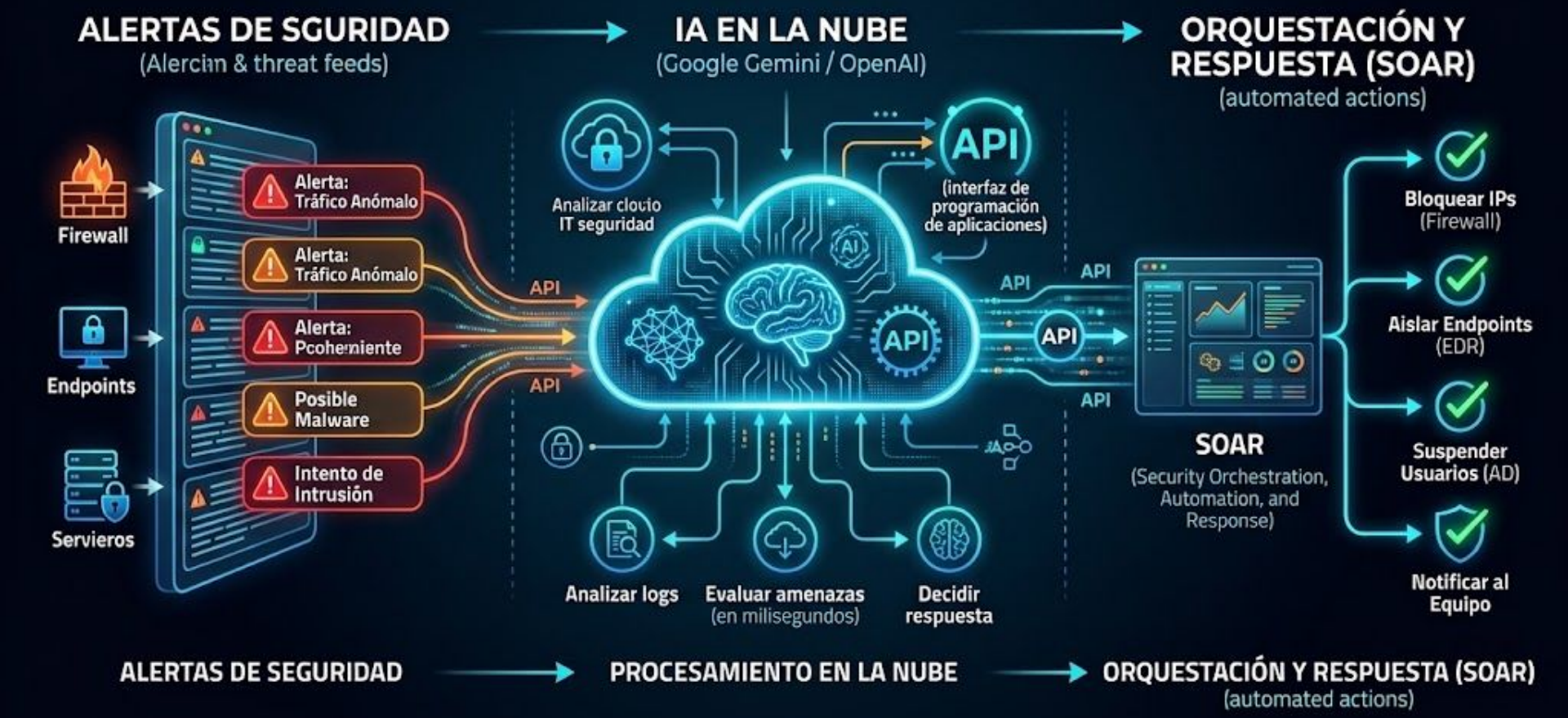
- Programación y diseño de Playbooks
- Entrenamiento y supervisión de modelos IA
- Optimización de la infraestructura SOC

La IA no reemplaza al humano en el SOC: automatiza las tareas repetitivas para que los analistas se enfoquen en la investigación y estrategia avanzada.

IA LOCAL & API EN RESPUESTA A INCIDENTES



IA EN LA NUBE & API EN RESPUESTA A INCIDENTES



ARQUITECTURA DE SOC CON ELASTIC (ELK) STACK

ORÍGENES DE DATOS

Endpoints
(Windows, Linux)

Red y Seguridad
(Firewalls, DNS, AD)

Nube y SaaS
(AWS, Azure, M365)

RECOLECCIÓN E INGESTA

Elastic Agent
(EDR básico)

Beats
(Winlogbeat, Filebeat)

Logstash
Filtros
(Filtros, Enriquecimiento y Normalización)

ALMACENAMIENTO Y CORRELACIÓN

Elasticsearch
(Motor de búsqueda y analítica)



Enriquecimiento
(Threat Intel - CTI, ECS)

VISUALIZACIÓN Y DETECCIÓN
(SIEM & Analytics)

Elastic Security & Kibana

Kibana Dashboards

Motor SIEM
(Reglas MITRE ATT&CK)

Machine Learning

Gestión de Casos

RESPUESTA Y AUTOMATIZACIÓN
(SOAR & Incident Response)

SOAR & APIs

- Aislar Endpoint
- Notificaciones (Slack, Jira, Teams)
- Bloquear IP en Firewall
- Bloquear Usuario (AD)

ARQUITECTURA DE SOC CON WAZUH

ORÍGENES DE DATOS



Endpoints & Servidores
(Windows, Linux, macOS)



Contenedores y Cloud
(Kubernetes, AWS, Azure)



Infraestructura de Red
(Firewalls via Syslog/Agentless)

RECOLECCIÓN

Wazuh Agent

Instalado para enformino son endpoints:

- FIM (Integridad de archivos)
- SCA (Configuración)
- Detección de Malware & Logs

Agentless / Syslog



Dispositivos de red
(Fierpernetes)

ANÁLISIS Y DETECCIÓN

Wazuh Server



- Motor de Reglas y Decodificación
- MITRE ATT&CK / Cumplimiento (PCI, GDPR)
- Detección de Vulnerabilidades (CVE)
- Threat Intelligence (VirusTotal)

ALMACENAMIENTO

Wazuh Indexer



Almazen eventos, alertas y índexos logos históricos

VISUALIZACIÓN Y GESTIÓN

Wazuh Dashboard



- Monitoreo de Alertas
- Inventario y Seguridad
- Paneles de Cumplimiento

RESPUESTA Y ORQUESTACIÓN

Respuesta Activa (Active Response) & SOAR



Ejecutar acciones en Agente
(Bloquear IP, matar proceso, aislar host)



Notificaciones e Integraciones
(Slack, Jira, Teams, SOAR Open Source Shuffle)

ARQUITECTURA DE SOC CON SPLUNK

ORÍGENES DE DATOS

Endpoints y Servidores
(Windows, Linux, macOS, EDR)

Infraestructura de Red
(Firewalls, Syslog, IDS)

Nube y SaaS
(AWS, Azure, M365)

RECOLECCIÓN E INGESTA

Universal Forwarder
(Agente ligero para endpoints)

Heavy Forwarder
(Filtrado y enrutamiento)

HTTP Event Collector (HEC) / Syslog
(APIs, logs de red)

ALMACENAMIENTO E INDEXACIÓN

Splunk Indexers



Splunk Indexers
Compress and index datos into buckets

Enriquecimiento
(Common Information Model – CIM)

BÚSQUEDA Y SIEM

Search Head & Splunk Enterprise Security (ES)

Kibana/Splunk Dashboards & Search UI

Motor de Correlación
(SIEM)

Risk-Based Alerting
(RBA)

Threat Intelligence

ORQUESTACIÓN Y RESPUESTA

Splunk SOAR

Aislar Endpoint
(con EDR)

Notificaciones e Incidentes
(Slack, Teams, Jira)

Bloquear IP en Firewall

Suspender Usuario
(AD)

ARQUITECTURA DE SOC CON IBM QRADAR

ORÍGENES DE DATOS

Endpoints y Servidores
(Windows, Linux, macOS, EDR)

Infraestructura de Red
(Firewalls, Syslog, QFlow/NetFlow)

Nube y SaaS
(AWS, Azure, Cloud, SaaS)

RECOLECCIÓN E INGESTA

Event Collector
(QRadar Event Collector)

Flow Collector
(QRadar QFlow Collector para tráfico de red)

WinCollect / Syslog
(Agentes y reenvío de logs)

PROCESAMIENTO Y CORRELACIÓN

QRadar Event & Flow Processor

Parsen datos,
parsen, normalizar y correlatan

Enriquecimiento
(DSM - Device Support Modules, Threat Intelligence)

ANÁLISIS Y GESTIÓN

QRadar Console & SIEM

QRadar Dashboard & Analyst UI

Motor de Correlación
(ARI - Offense Manager)

Análisis de Comportamiento
(UBA)

Investigación con IA
(QRadar Advisor with Watson)

RESPUESTA Y ORQUESTACIÓN

QRadar SOAR

Mitigación Activa
(Aislar endpoint)

Notificaciones e Incidentes
(Slack, Teams, Jira)

Bloquear IP en Firewall

Inhabilitar Usuario
(AD/IAM)

ARQUITECTURA DE SOC CON MICROSOFT SENTINEL



ARQUITECTURA DE SOC CON GOOGLE SECURITY OPERATIONS

ORÍGENES DE DATOS

Endpoints y Servidores
(Windows, Linux, macOS, EDR)

Infraestructura On-Premises
(Firewalls, Syslog, Directorio Activo)

Entornos Multi-Cloud y SaaS
(Google Cloud, AWS, Azure, Microsoft 365)

RECOLECCIÓN E INGESTA

Chronicle Forwarder

InGestión por API y Webhooks

Ingesta Serverless
(Cloud Logging, Pub/Sub)

ALMACENAMIENTO Y DETECCIÓN

Chronicle Data Lake / BigQuery

Normalización UDM
(Unified Data Model)

Enriquecimiento Automático

Retención de Datos
(Hot Data - 1 Año por Defecto)

Motor de Detección
(YARA-L)

Reglas de Detección
(YARA-L)

Inteligencia de Amenazas
(Mandiant & VirusTotal)

Machine Learning

INVESTIGACIÓN Y GESTIÓN

Google SecOps Portal



Gestión de Alertas y Casos

Búsquedas a Escala de Petabytes
(Velocidad de Búsqueda de Google)

Caza de Amenazas
(Threat Hunting)



ORQUESTACIÓN Y RESPUESTA

Google SecOps SOAR
(anteriormente Siemplify)

Playbooks y Automatización

Aislar Host
(con EDR)

Notificaciones y Tickets
Teamts
(Teams, Slack, Jira, ServiceNow)

Bloquear IP en Firewall

Inhabilitar Usuario
(AD / Cloud Identity)

DIMENSIONES DEL SOC-CMM

El modelo de madurez para Centros de Operaciones de Seguridad

NEGOCIO (Business)

Alineación con los objetivos de la organización, cumplimiento regulatorio, gestión de riesgos, presupuesto y gobernanza

PERSONAS (People)

Roles y responsabilidades, habilidades, entrenamiento, retención de talento y organización del equipo del SOC

PROCESOS (Processes)

Procedimientos operativos (SOP), gestión de incidentes, playbooks, mejora continua y flujos de trabajo

TECNOLOGÍA (Technology)

Herramientas de detección (SIEM), automatización (SOAR), recolección de logs, infraestructura y capacidades de búsqueda

SERVICIOS (Services)

Catálogo de servicios ofrecidos: monitoreo 24/7, caza de amenazas (hunting), respuesta a incidentes y análisis forense





NIVEL DE MADUREZ: Inicial (1) > Repetible (2) > Definido (3) > Gestionado (4) > Optimizado (5)

¿CÓMO ENTIENDE LA IA?



1. CONVIERTE PALABRAS EN NÚMEROS

Para la IA, la palabra “gato” no es un animal, es una lista de números (ej: 0.85, 0.12, 0.44)

2. BUSCA PATRONES Y PROBABILIDADES

No “entiende” el significado. Compara esos números con miles de millones de otros datos en su memoria matemática

3. SELECCIONA LA MEJOR RESPUESTA

Elige la combinación de palabras más probable en base a la matemática

¿CÓMO FUNCIONA EL PROGRAMA POR DENTRO?
(De palabras a números)

PASO 1: EL VOCABULARIO
(Diccionario base)



PASO 2: LA PREGUNTA SE HACE NÚMERO
(Vectorización)



que es gato

Ingresado

[1,1,0,1,0,0]

Basándose el diccionario

PASO 3: COMPARACIÓN MATEMÁTICA
(Similitud de Coseno)



Caja Matemática

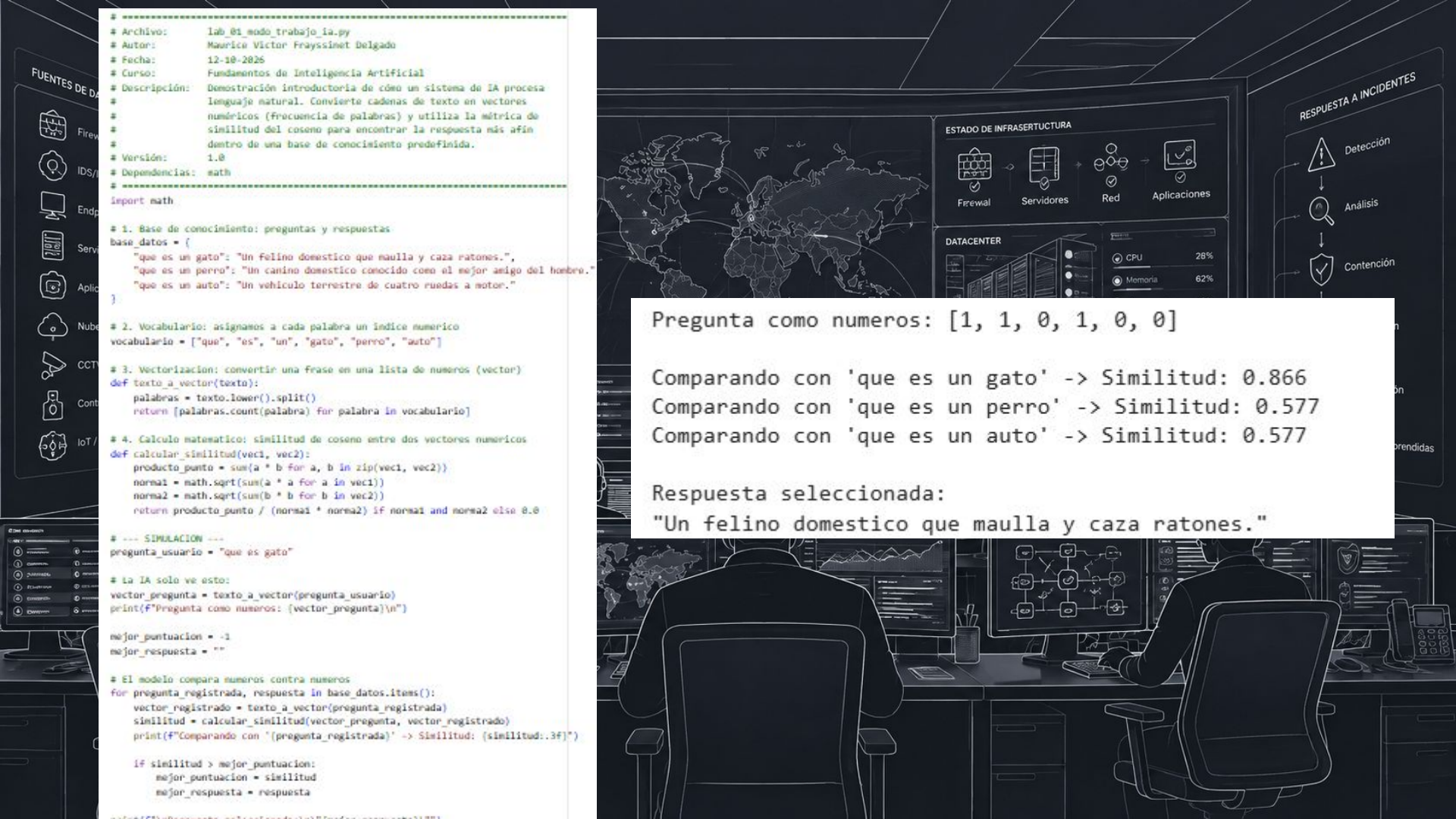
- "que es un gato" [1,1,1,1,0,0] → Similitud: 86.6% (¡Máxima!)
- "que es un perro" [1,1,1,0,1,0] → Similitud: 57.7%
- "que es un auto" [1,1,1,0,0,1] → Similitud: 57.7%

PASO 4: LA DECISIÓN
(Selección de respuesta)



"Un felino domestico que maulla y caza ratones."

¡La IA no entiende el significado, solo calcula cuál es la respuesta con la mayor puntuación matemática!




```python
# ============================================================================
# Archivo:        lab_01_modo_trabajo_ia.py
# Autor:          Maurice Victor Frayssinet Delgado
# Fecha:          12-10-2026
# Curso:          Fundamentos de Inteligencia Artificial
# Descripción:    Demostración introductoria de cómo un sistema de IA procesa
#                 lenguaje natural. Convierte cadenas de texto en vectores
#                 numéricos (frecuencia de palabras) y utiliza la métrica de
#                 similitud del coseno para encontrar la respuesta más afín
#                 dentro de una base de conocimiento predefinida.
# Versión:        1.0
# Dependencias:   math
# ============================================================================
import math

# 1. Base de conocimiento: preguntas y respuestas
base_datos = {
    "que es un gato": "Un felino domestico que maulla y caza ratones.",
    "que es un perro": "Un canino domestico conocido como el mejor amigo del hombre.",
    "que es un auto": "Un vehiculo terrestre de cuatro ruedas a motor."
}

# 2. Vocabulario: asignamos a cada palabra un índice numérico
vocabulario = ["que", "es", "un", "gato", "perro", "auto"]

# 3. Vectorización: convertir una frase en una lista de números (vector)
def texto_a_vector(texto):
    palabras = texto.lower().split()
    return [palabras.count(palabra) for palabra in vocabulario]

# 4. Cálculo matemático: similitud de coseno entre dos vectores numéricos
def calcular_similitud(vec1, vec2):
    producto_punto = sum(a * b for a, b in zip(vec1, vec2))
    norma1 = math.sqrt(sum(a * a for a in vec1))
    norma2 = math.sqrt(sum(b * b for b in vec2))
    return producto_punto / (norma1 * norma2) if norma1 and norma2 else 0.0

# --- SIMULACIÓN ---
pregunta_usuario = "que es gato"

# La IA solo ve esto:
vector_pregunta = texto_a_vector(pregunta_usuario)
print(f"Pregunta como numeros: {vector_pregunta}\n")

mejor_puntuacion = -1
mejor_respuesta = ""

# El modelo compara numeros contra numeros
for pregunta_registrada, respuesta in base_datos.items():
    vector_registrado = texto_a_vector(pregunta_registrada)
    similitud = calcular_similitud(vector_pregunta, vector_registrado)
    print(f"Comparando con '{pregunta_registrada}' -> Similitud: {similitud:.3f}")

    if similitud > mejor_puntuacion:
        mejor_puntuacion = similitud
        mejor_respuesta = respuesta
```

```
Pregunta como numeros: [1, 1, 0, 1, 0, 0]

Comparando con 'que es un gato' -> Similitud: 0.866
Comparando con 'que es un perro' -> Similitud: 0.577
Comparando con 'que es un auto' -> Similitud: 0.577

Respuesta seleccionada:
"Un felino domestico que maulla y caza ratones."
```

COMO LA IA CREA RESPUESTAS PALABRA POR PALABRA

(Autorregresión: la ruleta de probabilidades)

PASO 1: EL INICIO (EL PROMPT)



PASO 2: LA RULETA DE PROBABILIDADES



PASO 3: SE UNE AL TEXTO



PASO 4: EL BUCLE SE REPITE



PASO 1: EL INICIO (EL PROMPT)

El usuario escribe un inicio. La IA lo analiza para calcular qué podría venir después.

PASO 2: LA RULETA DE PROBABILIDADES

Calcula las probabilidades de la siguiente palabra basada en su entrenamiento. Elige la más probable.

PASO 3: SE UNE AL TEXTO

La palabra elegida se une al texto original, formando una nueva frase.

EL BUCLE SE REPITE

El proceso se repite con la nueva frase completa hasta generar un punto final o parada

La IA no piensa oraciones completas, ¡las calcula y las construye palabra por palabra usando estadística!




```
# ==========================================================
# Archivo:        lab_02_modo_traducir_to.py
# Autor:          Mauricio Ulises Procuniere Delgado
# Fecha:          12-10-2024
# Curso:          Fundamentos de Inteligencia Artificial
# Descripción:    Versión de visualidad (vuelo alianza) del motor de similitud
#                 de texto. Imprime en consola las operaciones matemáticas
#                 internas de la IA: mapeo del vocabulario base, cálculo del
#                 producto punto por a por, cálculo de normas (magnitudes) y
#                 la obtención del porcentaje de coincidencia de similitud del
#                 coseno para la toma de decisiones.
# Versión:        1.0
# Dependencias:   math
# ==========================================================
import math

# 1. Base de conocimientos
base_datos = {
    "que es un gato": "Un felino domestico que maulla y caza ratones.",
    "que es un perro": "Un canino domestico conocido como el mejor amigo del hombre.",
    "que es un auto": "Un vehiculo terrestre de cuatro ruedas a motor."
}

# 2. Diccionario o vocabulario base (posiciones 0 a 5)
vocabulario = ["que", "es", "un", "gato", "perro", "auto"]

print("\n" + "="*60)
print("PASO 1: VOCABULARIO BASE (MAPA DE PALABRAS A COORDENADAS)")
print("="*60)
for indice, palabra in enumerate(vocabulario):
    print(f"  Posición {indice} -> '{palabra}'")

# 3. Función de vectorización con visualidad
def texto_a_vector(texto, etiqueta=""):
    palabras = texto.lower().split()
    vector = [palabras.count(p) for p in vocabulario]
    if etiqueta:
        print(f"\n[Vectorización] {etiqueta}: \"{texto}\"")
        print(f"  Palabras detectadas: {palabras}")
        print(f"  Vector resultante  : {vector}")
    return vector

# 4. Función de similitud matemática con detalle de operaciones
def calcular_similitud_detallada(vec_pregunta, vec_candidato, nombre_candidato):
    print(f"\n--- Comparando con: \"{nombre_candidato}\" ---")
    print(f"  Vector pregunta   : {vec_pregunta}")
    print(f"  Vector candidato  : {vec_candidato}")

    # Producto punto: multiplicar posición por posición y sumar
    multiplicaciones = [p * c for p, c in zip(vec_pregunta, vec_candidato)]
    producto_punto = sum(multiplicaciones)
    print(f"  Multiplicación par a par: {multiplicaciones}")
    print(f"  Producto punto (suma)   : {producto_punto}")

    # Normas (magnitud de cada vector)
    norma_p = math.sqrt(sum(p ** 2 for p in vec_pregunta))
    norma_c = math.sqrt(sum(c ** 2 for c in vec_candidato))
    print(f"  Longitud vector pregunta : √({sum(p*p for p in vec_pregunta)}) = {norma_p:.4f}")
    print(f"  Longitud vector candidato: √({sum(c*c for c in vec_candidato)}) = {norma_c:.4f}")

    # Similitud de coseno: producto punto / (norma_p * norma_c)
    similitud = producto_punto / (norma_p * norma_c)
    print(f"  Cálculo final: {producto_punto} / ({norma_p:.4f} * {norma_c:.4f})")
    print(f"  -> Puntuación de similitud: {similitud:.4f} ({similitud * 100:.1f}%)")
    return similitud

# --- CONSOLA DEL PROGRAMA ---
pregunta_usuario = "que es gato"

print("\n" + "="*60)
print("PASO 2: CONVERTIR LA ENTRADA DEL USUARIO A NÚMEROS")
print("="*60)
vector_usuario = texto_a_vector(pregunta_usuario, "Entrada del usuario")

print("\n" + "="*60)
print("PASO 3: COMPARACIÓN MATEMÁTICA CON CADA OPCIÓN EN MEMORIA")
print("="*60)

mejor_puntuacion = -1
mejor_respuesta = ""
mejor_pregunta = ""

for pregunta_registrada, respuesta in base_datos.items():
    vector_registrado = texto_a_vector(pregunta_registrada)
    similitud = calcular_similitud_detallada(vector_usuario, vector_registrado, pregunta_registrada)

    if similitud > mejor_puntuacion:
        mejor_puntuacion = similitud
        mejor_respuesta = respuesta
        mejor_pregunta = pregunta_registrada

print("\n" + "="*60)
print("PASO 4: DECISIÓN DEL MODELO (MÁXIMA PROBABILIDAD / SIMILITUD)")
print("="*60)
print(f"Opción ganadora : \"{mejor_pregunta}\" con {mejor_puntuacion * 100:.1f}% de coincidencia.")
print(f"Respuesta final enviada al usuario:\n\"{mejor_respuesta}\"")
```

```
PASO 1: VOCABULARIO BASE (MAPA DE PALABRAS A COORDENADAS)
============================================================
  Posición 0 -> 'que'
  Posición 1 -> 'es'
  Posición 2 -> 'un'
  Posición 3 -> 'gato'
  Posición 4 -> 'perro'
  Posición 5 -> 'auto'

============================================================
PASO 2: CONVERTIR LA ENTRADA DEL USUARIO A NÚMEROS
============================================================

[Vectorización] Entrada del usuario: "que es gato"
  Palabras detectadas: ['que', 'es', 'gato']
  Vector resultante  : [1, 1, 0, 1, 0, 0]

============================================================
PASO 3: COMPARACIÓN MATEMÁTICA CON CADA OPCIÓN EN MEMORIA
============================================================

--- Comparando con: "que es un gato" ---
  Vector pregunta   : [1, 1, 0, 1, 0, 0]
  Vector candidato  : [1, 1, 1, 1, 0, 0]
  Multiplicación par a par: [1, 1, 0, 1, 0, 0]
  Producto punto (suma)   : 3
  Longitud vector pregunta : √3 = 1.7321
  Longitud vector candidato: √4 = 2.0000
  Cálculo final: 3 / (1.7321 * 2.0000)
  -> Puntuación de similitud: 0.8660 (86.6%)

--- Comparando con: "que es un perro" ---
  Vector pregunta   : [1, 1, 0, 1, 0, 0]
  Vector candidato  : [1, 1, 1, 0, 1, 0]
  Multiplicación par a par: [1, 1, 0, 0, 0, 0]
  Producto punto (suma)   : 2
  Longitud vector pregunta : √3 = 1.7321
  Longitud vector candidato: √4 = 2.0000
  Cálculo final: 2 / (1.7321 * 2.0000)
  -> Puntuación de similitud: 0.5774 (57.7%)

--- Comparando con: "que es un auto" ---
  Vector pregunta   : [1, 1, 0, 1, 0, 0]
  Vector candidato  : [1, 1, 1, 0, 0, 1]
  Multiplicación par a par: [1, 1, 0, 0, 0, 0]
  Producto punto (suma)   : 2
  Longitud vector pregunta : √3 = 1.7321
  Longitud vector candidato: √4 = 2.0000
  Cálculo final: 2 / (1.7321 * 2.0000)
  -> Puntuación de similitud: 0.5774 (57.7%)

============================================================
PASO 4: DECISIÓN DEL MODELO (MÁXIMA PROBABILIDAD / SIMILITUD)
============================================================
Opción ganadora : "que es un gato" con 86.6% de coincidencia.
Respuesta final enviada al usuario:
"Un felino domestico que maulla y caza ratones."
```

¿CÓMO ENTIENDE LA IA PALABRAS NUEVAS?

(**Embeddings:** El mapa matemático de los significados)

PASO 1: EL USUARIO ESCRIBE "FELINO"

felino

↓

convertida

[0.98, 0.60]

basados en su **significado biológico**, en vez de buscar letras exactas.

MAPA SEMÁNTICO (COORDEANDAS 2D)

NIVEL MASCOTA / DOMÉSTICO (0 a 1)

NIVEL ANIMAL (0 a 1)

michi (0.90, 0.95)

perro

gato (0.95, 0.90)

PASO 2: CÁLCULO DE DISTANCIA (Pitágoras)

gato (0.95, 0.90)

= 0.30

perro (0.92, 0.95)

= 0.35

= 1.09

¡La distancia más pequeña es hacia 'gato'!

felino (0.98, 0.60)

1.09

auto (0.01, 0.10)

camion (0.02, 0.05)

felino (0.98, 0.60)

PASO 3: LA RESPUESTA

Un mamífero carnívoro de la familia de los felinos...

Respuesta de **gato** seleccionada por cercanía semántica

La IA no entiende qué es un 'felino', pero sabe que en su mapa matemático está casi en el mismo lugar que un 'gato'

```python
# ============================================================================
# Archivo:        lab_03_generacion_texto_ia.py
# Autor:          Maurice Victor Frayssinet Delgado
# Fecha:          12-10-2026
# Curso:          Fundamentos de Inteligencia Artificial
# Descripción:    Implementación de un modelo de lenguaje generativo básico
#                 basado en Cadenas de Markov. Simula la fase de entrenamiento
#                 sobre un corpus de texto para construir una matriz de transición
#                 (probabilidades de secuencias de palabras) y demuestra la
#                 generación autorregresiva (predicción del siguiente token) paso
#                 a paso hasta encontrar un token de parada (.).
# Versión:        1.0
# Dependencias:   random, time
# ============================================================================
import random
import time

# 1. Datos de entrenamiento (lo que el modelo "lee" para aprender el mundo)
corpus = [
    "un gato es un felino que caza ratones .",
    "un gato es un animal domestico muy limpio .",
    "un gato toma leche y duerme en el sofa .",
    "el gato maulla cuando tiene hambre ."
]

# 2. Fase de entrenamiento: Aprender qué palabra sigue a cuál
modelo_probabilidades = {}

for oracion in corpus:
    tokens = oracion.split()
    for i in range(len(tokens) - 1):
        palabra_actual = tokens[i]
        palabra_siguiente = tokens[i + 1]

        if palabra_actual not in modelo_probabilidades:
            modelo_probabilidades[palabra_actual] = []
        modelo_probabilidades[palabra_actual].append(palabra_siguiente)

print("=" * 65)
print("PASO 1: MODELO ENTRENADO (MATRIZ DE TRANSICIÓN)")
print("=" * 65)
for origen, destinos in modelo_probabilidades.items():
    print(f"Tras '{origen}' suele venir: {destinos}")

# 3. Generación autorregresiva (token por token)
def generar_texto(palabra_inicial, max_palabras=10):
    frase = [palabra_inicial]
    palabra_actual = palabra_inicial

    print("\n" + "=" * 65)
    print(f"PASO 2: GENERACIÓN AUTORREGRESIVA (Inicio: '{palabra_inicial}')")
```

```
=================================================================
PASO 1: MODELO ENTRENADO (MATRIZ DE TRANSICIÓN)
=================================================================
Tras 'un' suele venir: ['gato', 'felino', 'gato', 'animal', 'gato']
Tras 'gato' suele venir: ['es', 'es', 'toma', 'maulla']
Tras 'es' suele venir: ['un', 'un']
Tras 'felino' suele venir: ['que']
Tras 'que' suele venir: ['caza']
Tras 'caza' suele venir: ['ratones']
Tras 'ratones' suele venir: ['.']
Tras 'animal' suele venir: ['domestico']
Tras 'domestico' suele venir: ['muy']
Tras 'muy' suele venir: ['limpio']
Tras 'limpio' suele venir: ['.']
Tras 'toma' suele venir: ['leche']
Tras 'leche' suele venir: ['y']
Tras 'y' suele venir: ['duerme']
Tras 'duerme' suele venir: ['en']
Tras 'en' suele venir: ['el']
Tras 'el' suele venir: ['sofa', 'gato']
Tras 'sofa' suele venir: ['.']
Tras 'maulla' suele venir: ['cuando']
Tras 'cuando' suele venir: ['tiene']
Tras 'tiene' suele venir: ['hambre']
Tras 'hambre' suele venir: ['.']

=================================================================
PASO 2: GENERACIÓN AUTORREGRESIVA (Inicio: 'un')
=================================================================

[Paso 1] Contexto actual: un
  Candidatos posibles: {'gato': '60.0%', 'animal': '20.0%', 'felino': '20.0%'}
  -> Palabra seleccionada: 'gato'

[Paso 2] Contexto actual: un gato
  Candidatos posibles: {'maulla': '25.0%', 'es': '50.0%', 'toma': '25.0%'}
  -> Palabra seleccionada: 'maulla'

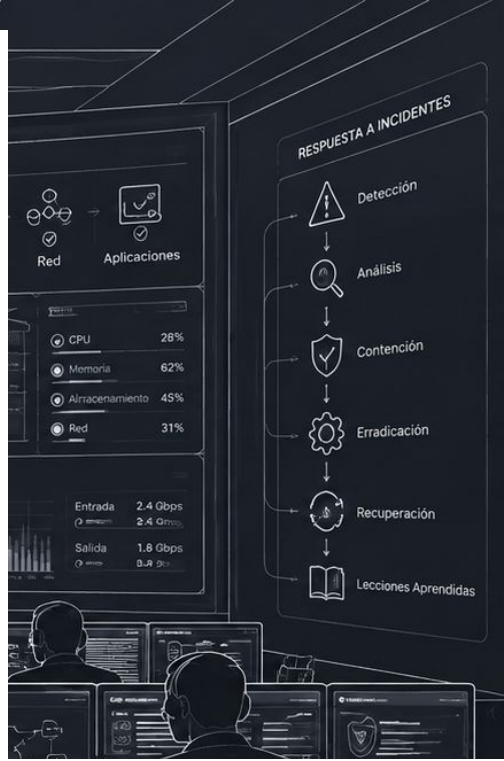
[Paso 3] Contexto actual: un gato maulla
  Candidatos posibles: {'cuando': '100.0%'}
  -> Palabra seleccionada: 'cuando'

[P
[P
[P
  -> Palabra seleccionada: '.'

[Fin detectado: Se generó el token de parada '.']

=================================================================
TEXTO FINAL SINTETIZADO POR LA IA:
```

```
=================================================================
TEXTO FINAL SINTETIZADO POR LA IA:
=================================================================
"un gato maulla cuando tiene hambre ."
```



¿QUÉ ES UN MODELO "SYSTEM ONE" IN IA?

Es una nueva clase de IA diseñada exclusivamente para tomar decisiones **ultra rápidas y estructuradas en milisegundos, sin generar texto**

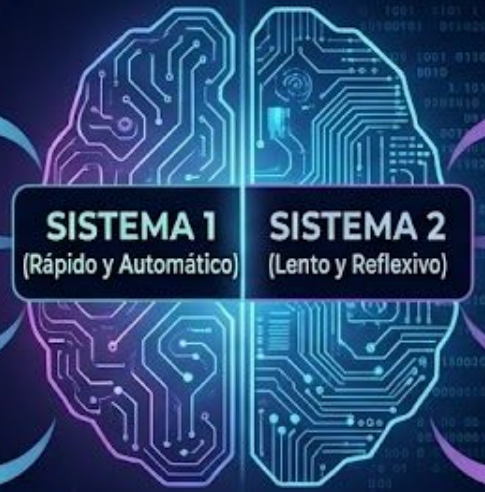
Inspirados en la psicología cognitiva de psitología cognitiva de Daniel Kahneman:



SISTEMA 1
(Rápido y Automático)

Jev y Laya

- Decisión en milisegundos (70-500ms)
- Opciones cerradas y probabilidades
- Sin "alucinaciones" de texto
- Ideal para clasificar, filtrar y enrutar



- Razonamiento complejo
- Generación de texto y diálogo
- Segundos o minutos por respuesta
- Ideal para redactar, programar y analizar



SISTEMA 2
(Lento y Reflexivo)

los LLM tradicionales (con ChatGPT, Claude)



SIN GENERACIÓN DE TEXTO

No escriben respuestas; solo eligen la mejor opción o la probabildad de una lista cerrada.



VELOCIDAD EXTREMA

Respuestas casi instantáneas (sub-segundo) ideales para automatización en tiempo real y agentes.

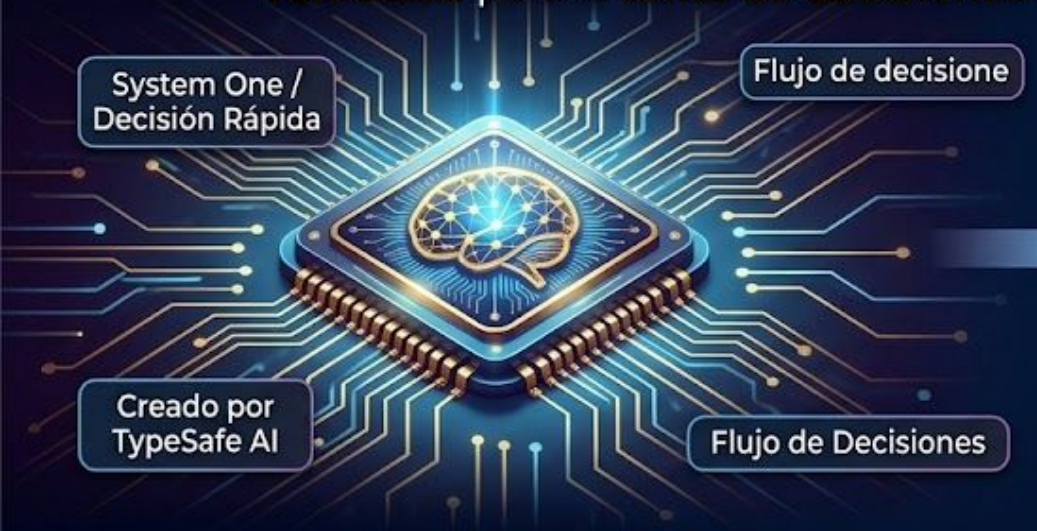


EFICIENCIA Y BAJO COSTO

Requieren muchísima menos capacidad de cómputo que los modelos generativos pesados.

¿QUÉ ES JEV IA?

Jev IA, la inteligencia artificial de tipo **'System One'** de **baja latencia** y **alta velocidad** para la **toma de decisiones** estructuradas en milisegundos.



Input
(datos, preguntas)

Output

Instantáneo
(opciones cerradas con probabilidad matemática de acierto)

70ms - 500ms

TOMA DE DECISIONES SIN TEXTO

Elige opciones predefinidas de listas cerradas (como sí/no, clasificación, urgencia, fraude) en lugar de redactar, eliminando las alucinaciones.

BAJA LATENCIA Y ALTO RENDIMIENTO

Diseñado para operaciones automáticas en tiempo real en sistemas críticos y agentes de IA ae que requieren respuestas inmediatas.

API GESTIONADA EN LA NUBE

Un servicio robusto de pago alojado por TypeSafe AI, optimizado para integrarse de forma sencilla y eficiente en aplicaciones empresariales.

¿QUÉ ES LAYA IA?

El modelo System One de código abierto para **la toma de decisiones rápidas, locales y seguras**



Ejecución Local (Offline)
Creado por **Convai Innovations**
(Licencia Apache 2.0)

Flujo de decisión

Input (datos, preguntas)

Velocidad ultra rápida

Output Instantáneo
(Opciones cerradas con probabilidad matemática de acierto)

CÓDIGO ABIERTO (OPEN SOURCE)

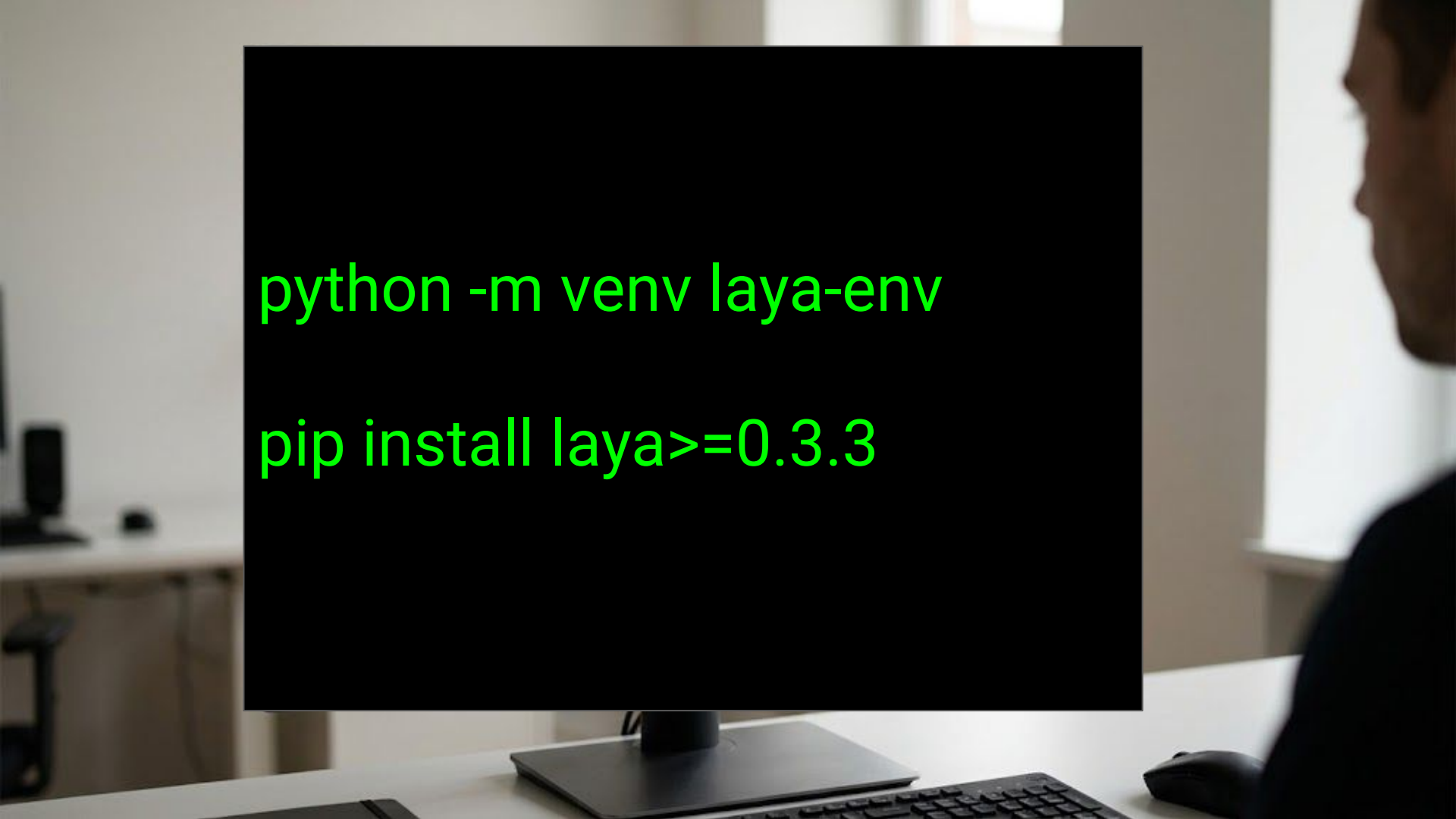
Publicado bajo licencia Apache 2.0. Permite a los desarrolladores acceder, modificar y ser dueños de los pesos del modelo

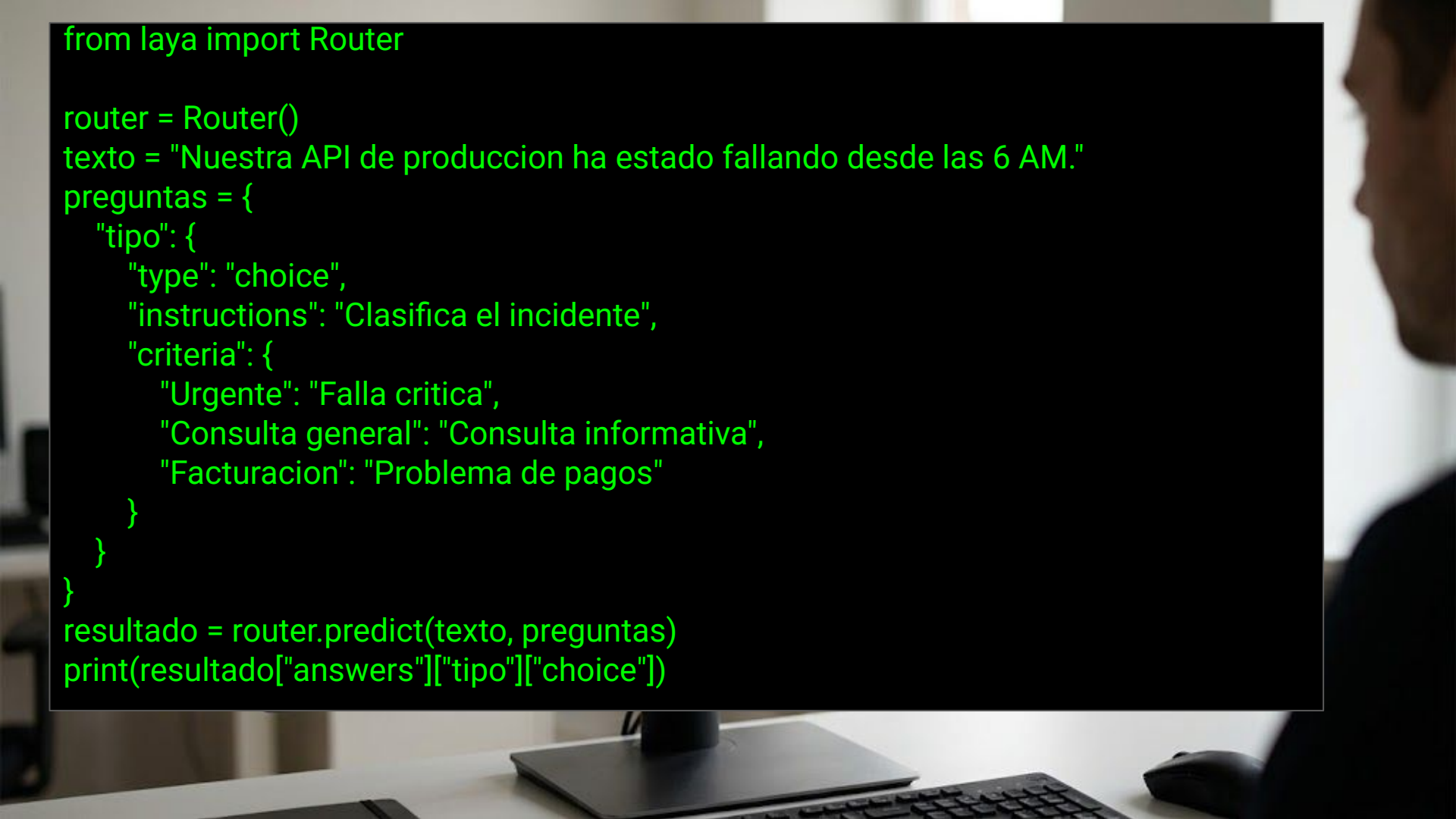
SISTEMA LOCAL Y PRIVADO

Funciona offline en tu propia infraestructura. Ideal para empresas que requieren máxima privacidad y soberanía de sus datos

TOMA DE DECISIONES SIN TEXTO

Diseñado como "System One" para tareas automatizadas de clasificación, filtrado y agentes, sin generar texto o "alucinaciones"


python -m venv laya-env
pip install laya>=0.3.3



```
from laya import Router

router = Router()
texto = "Nuestra API de produccion ha estado fallando desde las 6 AM."
preguntas = {
    "tipo": {
        "type": "choice",
        "instructions": "Clasifica el incidente",
        "criteria": {
            "Urgente": "Falla critica",
            "Consulta general": "Consulta informativa",
            "Facturacion": "Problema de pagos"
        }
    }
}
resultado = router.predict(texto, preguntas)
print(resultado["answers"]["tipo"]["choice"])
```

```
import csv
import json
from pathlib import Path
from collections import Counter, defaultdict
from datetime import datetime, timedelta

BASE = Path(__file__).resolve().parent
LOG = BASE / "eventos_demo.csv"

def cargar():
    if not LOG.exists():
        crear_demo()
    with LOG.open(encoding="utf-8", newline="") as f:
        return list(csv.DictReader(f))

def crear_demo():
    filas = []
    inicio = datetime(2026, 10, 14, 8, 0)
    for i in range(120):
        tipo = ["login_ok", "login_fail", "firewall_deny", "web_request", "port_connection"][i % 5]
        ip = "203.0.113.25" if i % 7 == 0 else f"192.0.2.{(i % 15) + 1}"
        detalle = "normal"
        if tipo == "web_request" and i % 3 == 0: detalle = "GET /?id=1 UNION SELECT password FROM users"
        if tipo == "port_connection": detalle = f"dst_port={20 + i}"
        if tipo == "firewall_deny": detalle = "blocked tcp connection"
        filas.append(dict(timestamp=(inicio + timedelta(minutes=i*8)).isoformat(), ip=ip, usuario=f"user{i%8}", evento=tipo, detalle=detalle))
    for j in range(15):
        filas.append(dict(timestamp=(inicio+timedelta(minutes=j)).isoformat(),ip="198.51.100.77",usuario="admin",evento="login_fail",detalle="bad password"))
    for j in range(30):
        filas.append(dict(timestamp=(inicio+timedelta(seconds=j*10)).isoformat(),ip="198.51.100.88",usuario="-",evento="port_connection",detalle=f"dst_port={1000+j}"))
    with LOG.open("w", encoding="utf-8", newline="") as f:
        w=csv.DictWriter(f, fieldnames=["timestamp","ip","usuario","evento","detalle"]); w.writeheader(); w.writerows(filas)
    print("Dataset de demostraci├│n creado:", LOG)

if __name__ == "__main__":
    eventos = cargar()
    fallos = Counter((e["ip"], e["usuario"]) for e in eventos if e["evento"] == "login_fail")
    for (ip, usuario), n in fallos.most_common():
        if n >= 5: print(f"ALERTA fuerza bruta: IP={ip}, usuario={usuario}, fallos={n}")
```

```
import csv
import json
from pathlib import Path
from collections import Counter, defaultdict
from datetime import datetime, timedelta

BASE = Path(__file__).resolve().parent
LOG = BASE / "eventos_demo.csv"

def cargar():
    if not LOG.exists():
        crear_demo()
    with LOG.open(encoding="utf-8", newline="") as f:
        return list(csv.DictReader(f))

def crear_demo():
    filas = []
    inicio = datetime(2026, 10, 14, 8, 0)
    for i in range(120):
        tipo = ["login_ok", "login_fail", "firewall_deny", "web_request", "port_connection"][i % 5]
        ip = "203.0.113.25" if i % 7 == 0 else f"192.0.2.{(i % 15) + 1}"
        detalle = "normal"
        if tipo == "web_request" and i % 3 == 0: detalle = "GET /?id=1 UNION SELECT password FROM users"
        if tipo == "port_connection": detalle = f"dst_port={20 + i}"
        if tipo == "firewall_deny": detalle = "blocked tcp connection"
        filas.append(dict(timestamp=(inicio + timedelta(minutes=i*8)).isoformat(), ip=ip, usuario=f"user{i%8}", evento=tipo, detalle=detalle))
    for j in range(15):
        filas.append(dict(timestamp=(inicio+timedelta(minutes=j)).isoformat(),ip="198.51.100.77",usuario="admin",evento="login_fail",detalle="bad password"))
    for j in range(30):
        filas.append(dict(timestamp=(inicio+timedelta(seconds=j*10)).isoformat(),ip="198.51.100.88",usuario="-",evento="port_connection",detalle=f"dst_port={1000+j}"))
    with LOG.open("w", encoding="utf-8", newline="") as f:
        w=csv.DictWriter(f, fieldnames=["timestamp","ip","usuario","evento","detalle"]); w.writeheader(); w.writerows(filas)
    print("Dataset de demostraci├│n creado:", LOG)

if __name__ == "__main__":
    eventos=cargar()
    puntajes=Counter()
    for e in eventos:
        puntajes[e["ip"]] += {"login_fail":3,"firewall_deny":2,"port_connection":1}.get(e["evento"],0)
    for ip, puntos in puntajes.most_common(10): print(f"{ip}: riesgo heur├¡stico={puntos}")
```

```
import csv
import json
from pathlib import Path
from collections import Counter, defaultdict
from datetime import datetime, timedelta

BASE = Path(__file__).resolve().parent
LOG = BASE / "eventos_demo.csv"

def cargar():
    if not LOG.exists():
        crear_demo()
    with LOG.open(encoding="utf-8", newline="") as f:
        return list(csv.DictReader(f))

def crear_demo():
    filas = []
    inicio = datetime(2026, 10, 14, 8, 0)
    for i in range(120):
        tipo = ["login_ok", "login_fail", "firewall_deny", "web_request", "port_connection"][i % 5]
        ip = "203.0.113.25" if i % 7 == 0 else f"192.0.2.{(i % 15) + 1}"
        detalle = "normal"
        if tipo == "web_request" and i % 3 == 0: detalle = "GET /?id=1 UNION SELECT password FROM users"
        if tipo == "port_connection": detalle = f"dst_port={20 + i}"
        if tipo == "firewall_deny": detalle = "blocked tcp connection"
        filas.append(dict(timestamp=(inicio + timedelta(minutes=i*8)).isoformat(), ip=ip, usuario=f"user{i%8}", evento=tipo, detalle=detalle))
    for j in range(15):
        filas.append(dict(timestamp=(inicio+timedelta(minutes=j)).isoformat(),ip="198.51.100.77",usuario="admin",evento="login_fail",detalle="bad password"))
    for j in range(30):
        filas.append(dict(timestamp=(inicio+timedelta(seconds=j*10)).isoformat(),ip="198.51.100.88",usuario="-",evento="port_connection",detalle=f"dst_port={1000+j}"))
    with LOG.open("w", encoding="utf-8", newline="") as f:
        w=csv.DictWriter(f, fieldnames=["timestamp","ip","usuario","evento","detalle"]); w.writeheader(); w.writerows(filas)
    print("Dataset de demostraci├│n creado:", LOG)

if __name__ == "__main__":
    eventos=cargar()
    conteo=Counter(e["ip"] for e in eventos if e["evento"]=="firewall_deny")
    print("Conexiones bloqueadas por IP:")
    for ip,n in conteo.most_common(): print(ip,n)
```

```
C:\laya>python lab_01_laya.py
Dataset de demostración creado: C:\laya\eventos_demo.csv
ALERTA fuerza bruta: IP=198.51.100.77, usuario=admin, fallos=15

C:\laya>python lab_02_laya.py
198.51.100.77: riesgo heurístico=45
198.51.100.88: riesgo heurístico=30
203.0.113.25: riesgo heurístico=21
192.0.2.2: riesgo heurístico=21
192.0.2.7: riesgo heurístico=21
192.0.2.12: riesgo heurístico=21
192.0.2.3: riesgo heurístico=14
192.0.2.13: riesgo heurístico=14
192.0.2.8: riesgo heurístico=12
192.0.2.5: riesgo heurístico=7

C:\laya>python lab_03_laya.py
Conexiones bloqueadas por IP:
192.0.2.3 7
192.0.2.13 7
192.0.2.8 6
203.0.113.25 4
```

DESCARGA LOS EJEMPLOS Y LA PRESENTACIÓN



CONTACTO

Maurice Frayssinet
mfrayssinet@gmail.com
Tlf.: +51 980.997.203

